

1. Problem Summary

Product X has 100 units available, and about 10,000 customers will press "Buy Now" at the same moment. The platform must stay available under the surge, never sell more than 100 units, charge each customer at most once, and ensure every paid customer ends up with a valid order, even when services or the payment gateway fail.

2. Functional Requirements

Module	Requirement
Product	Customer can view product, price and stock
Cart	Customer can add the product to cart.
Reservation	System reserves stock when customer buys
Payment	System processes payments
Order	Successful payment creates an order
Reservation	Expired or failed reservations are released
Idempotency	Repeated requests do not create duplicate transactions
Shipment	Customer can track delivery
Notification	Customer receives important order and payment updates

3. Non-Functional Requirements

Each requirement is marked as a Guarantee (must always hold, even under failure) or a Target (best-effort goal that may degrade under extreme load).

Category	Requirement	Type
Inventory	Never sell more than 100 units.	Guarantee
Idempotency	Same request cannot create duplicate transactions.	Guarantee
Payment	Customer must not be charged twice	Guarantee
Scalability	System should handle high traffic.	Target
Security	Purchase APIs require authentication	Guarantee
Availability	System should remain available during the sale	Target
Observability	System should provide logs, metrics and traces	Target

Guarantees vs Targets

Strict guarantees: no overselling and no negative stock; no duplicate reservation, payment or order for the same request; no double charge and no paid customer without an order or refund; durable, auditable inventory, payment and order records; authenticated and authorised purchase APIs.

Targets: latency and availability figures, timing of expired-reservation release, order creation time after payment, and notification delivery speed.